

Digital Design & Computer Arch.

Problem Solving V

Prof. Onur Mutlu

ETH Zürich
Spring 2026
24 July 2026

Problem Solving V

- We solve 12 questions from Final Exam Spring 2021
 - ❑ Boolean Logic Circuits (Q1)
 - ❑ Verilog (Q2)
 - ❑ Finite State Machine (Q3)
 - ❑ ISA vs. Microarchitecture (Q4)
 - ❑ Performance Evaluation (Q5)
 - ❑ Pipelining (Q6)
 - ❑ Tomasulo's Algorithm (Q7)
 - ❑ GPUs and SIMD (Q8)
 - ❑ Branch Prediction (Q9)
 - ❑ Caches (Q10)
 - ❑ Prefetching (Q11)
 - ❑ Systolic Arrays (Q12)

Boolean Logic Circuits

(Q1)

- a) Using Boolean algebra, find the simplest Boolean algebra equation for the following min-terms:
 $\Sigma(1111, 1110, 1000, 1001, 1011, 1010, 0000)$. Show your work step-by-step.

- b) Convert the following Boolean equation so that it only contains NOR operations. Show your work step-by-step.

$$F = \overline{A} + \overline{(B.C + \overline{A.C})}$$

2.1 Complete the Verilog Code

For each numbered blank ①-⑤ in the following Verilog code, **mark the choice below** (i.e., one of options A, B, C, D) that makes the Verilog module operate as described in the comments. The resulting code must have correct syntax.

```
1 module my_module (input clk, input rst,
2   input [15:0] idata, input [1:0] op, ① [31:0] odata);
3
4   ② nval = 32'd0; // defining a 32-bit signal with an initial value of 0
5
6   always@* begin
7     case (op)
8       2'b00:
9         nval = odata + idata; // when 'op' is decimal 0, add 'idata' to
10                                // 'odata' and assign the result to 'nval'
11       2'b01:
12         nval = odata - idata; // when 'op' is decimal 1, subtract 'idata'
13                                // from 'odata' and assign the result to 'nval'
14       2'b10:
15         nval = idata; // when 'op' is decimal 2, assign 'idata' to 'nval'
16       ③:
17         nval = 0; // when 'op' is decimal 3, assign 0 to 'nval'
18     endcase
19   end
20
21   // executing the following always block on the rising edge of 'clk'
22   always@ (posedge clk) begin
23     if (rst)
24       ④ // resetting 'odata' to 0 for the next cycle
25     else
26       ⑤ // assigning 'nval' to 'odata' for the next cycle
27   end
28 endmodule
```

Provide your choice for each blank ①-⑤ below:

- | | | | |
|-----------------------------|-------------------------|-------------------|-------------------|
| ①: A. output | B. output reg | C. output wire | D. input reg |
| ②: A. reg[31:0] | B. input[31:0] | C. wire[31:0] | D. int[31:0] |
| ③: A. 2'b3 | B. 3'b3 | C. 2'h11 | D. default |
| ④: A. assign odata <= 0; | B. assign odata = 0; | C. odata == 0; | D. odata <= 0; |
| ⑤: A. assign odata <= nval; | B. assign odata = nval; | C. odata == nval; | D. odata <= nval; |

2.2 What Does This Code Do?

You are given a Verilog code that you are asked to analyze and find out what it does.

```
1  module my_module2 (input clk, output[1:0] out);
2
3      reg state = 1'b0;
4      reg[1:0] my_reg = 0;
5
6      always@(posedge clk) begin
7          state <= &out ? ~state : state;
8      end
9
10     always@(posedge clk) begin
11         case(state)
12             1'b0: begin
13                 my_reg <= my_reg + 1;
14             end
15             1'b1: begin
16                 my_reg <= my_reg - 1;
17             end
18         endcase
19     end
20
21     assign out = my_reg;
22 endmodule
```

Show the values (as unsigned decimal numbers) that the out signal takes, starting from the initial state of the module, for 16 consecutive clock (i.e., clk) cycles. Explain your answer briefly.

Finite State Machines

(Q3)

3.1 Simplifying an FSM

You are given the Mealy state machine of a *one input / one output* digital circuit design. Answer the following questions for the given state diagram.

- (a) Is it possible to simplify this state diagram and reduce the number of states?

If so, simplify it to the minimum number of states.

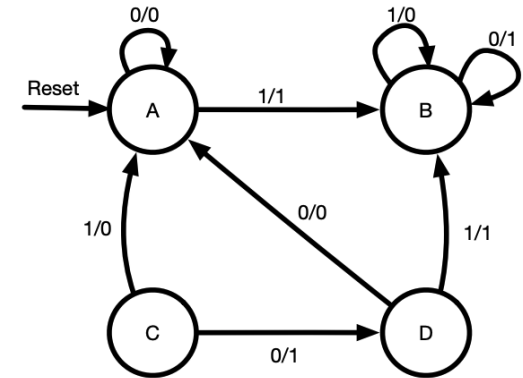
Explain each step of your simplification.

Draw the simplified state diagram.

If not, explain why it is not possible to simplify the state diagram.

- (b) Assume this state machine is used to process binary numbers from the least significant bit to the most significant bit. You are given an input bit stream: "10110100".

Please show the output bit stream produced by this FSM.



3.2 Designing an FSM

Design a Moore finite state machine (FSM) with one input and one output. The input provides an unsigned binary number in a bit-serial fashion from the most-significant bit to the least-significant bit. The output should be logic-1 in a clock cycle if the provided input so far is divisible by 8 (i.e., $[\text{the input number}] \bmod 8 = 0$). (Hint: Recall that the output depends only on the current state in a Moore FSM.)

Below are some example bit-streams that should output a logic-1 value.

- 1000
- 10000
- 11000
- 111000
- 101000

To start an input bit stream, the user should reset the FSM. Draw the state diagram and explain why it works. Your state machine should use as few states as possible and each state should have a precise definition and output.

ISA vs. Microarchitecture

(Q4)

A new CPU has two comprehensive user manuals available for purchase which describe the ISA and the microarchitecture of the CPU, respectively. Unfortunately, the manuals are extremely expensive, and you can only afford one of the two. If both manuals might be useful, you would prefer the ISA manual since it is much cheaper than the microarchitecture manual. For each of the following questions that you would like to answer, decide which manual is more likely to help. *Note: we will subtract 1 point for each incorrect answer and award 0 points for unanswered questions (the minimum number of total points you can get for this question is 0).*

- | | | |
|---|--------|----------------------|
| 1. Number of uniquely identifiable memory locations. | 1. ISA | 2. Microarchitecture |
| 2. Number of instructions fetched per clock cycle. | 1. ISA | 2. Microarchitecture |
| 3. Support for branch prediction hints conveyed by the compiler. | 1. ISA | 2. Microarchitecture |
| 4. Number of general-purpose registers. | 1. ISA | 2. Microarchitecture |
| 5. Number of non-programmable registers. | 1. ISA | 2. Microarchitecture |
| 6. SIMD processing support. | 1. ISA | 2. Microarchitecture |
| 7. Number of integer arithmetic and logic units (ALUs). | 1. ISA | 2. Microarchitecture |
| 8. Number of read ports in the physical register file. | 1. ISA | 2. Microarchitecture |
| 9. Endianness (big endian vs. small endian). | 1. ISA | 2. Microarchitecture |
| 10. Size of a virtual memory page. | 1. ISA | 2. Microarchitecture |
| 11. Cache coherence protocol. | 1. ISA | 2. Microarchitecture |
| 12. Number of cache blocks in the L3 cache. | 1. ISA | 2. Microarchitecture |
| 13. Ability to flush (i.e., invalidate) a cache line using the operating system code. | 1. ISA | 2. Microarchitecture |
| 14. Number of pipeline stages. | 1. ISA | 2. Microarchitecture |
| 15. How many prefetches the hardware prefetcher generates in a clock cycle. | 1. ISA | 2. Microarchitecture |

Performance Evaluation

(Q5)

A multi-cycle processor $P1$ executes *load instructions* in 6 cycles, *store instructions* in 6 cycles, *arithmetic instructions* in 2 cycles, and *branch instructions* in 2 cycles. Consider an application A where 40% of all instructions are load instructions, 20% of all instructions are store instructions, 30% of all instructions are arithmetic instructions, and 10% of all instructions are branch instructions.

- (a) What is the CPI of application A when executing on processor $P1$? Show your work.
- (b) A new design of the processor doubles the clock frequency of $P1$. However, the latencies of *all* instructions increase by 4 cycles. We call this new processor $P2$. The compiler used to generate instructions for $P2$ is the same as for $P1$. Thus, it produces the same number of instructions for program A . What is the CPI of application A when executing on processor $P2$? Show your work.
- (c) Which processor is faster ($P1$ or $P2$)? By how much (i.e., what is the speedup)? Show your work.
- (d) You want to improve the original $P1$ design by including one new optimization without changing the clock frequency. You can choose only one of the following options:
 - (1) **ALU**: An optimized *ALU*, which *halves* the latency of both arithmetic and branch instructions.
 - (2) **LSU**: An *asymmetric load-store unit*, which *halves* the latency of load operations but *doubles* the latency of store operations.Which optimization do you add to $P1$ for application A ? Show your work and justify your choice.

Pipelining

(Q6)

Consider two pipelined machines implementing the MIPS ISA, Machine A and Machine B. Both machines have *one* ALU and the following *five pipeline stages*, very similar to the basic 5-stage pipelined MIPS processor we discussed in lectures:

1. Fetch (one clock cycle)
2. Decode (one clock cycle)
3. Execute (one clock cycle)
4. Memory (one clock cycle)
5. Write-back (one clock cycle)

Machines A and B have the following specifications:

Consider the following code segment:

```
Loop: lw    $1, 0($4)
      lw    $2, 400($4)
      add   $3, $1, $2
      sw    $3, 0($4)
      sub   $4, $4, #4
      bnez  $4, Loop
```

	Machine A	Machine B
Data Forwarding/Interlocking	Does NOT implement interlocking in hardware. Relies on the compiler to order instructions or insert nop instructions such that dependent instructions are correctly executed.	Implements data dependence detection and data forwarding in hardware. On detection of instruction dependence, it forwards an operand from the memory stage or from the write-back stage to the execute stage. The result of a load instruction (lw) can <i>only</i> be forwarded from the write-back stage.
Internal register file forwarding	Implemented (i.e., an instruction writes into a register in the first half of a cycle and another instruction can correctly access the same register in the second half of the cycle).	Same as Machine A
Branch Prediction	Predicts all branches as <i>always-taken</i> , and the next program counter is available after the decode stage.	Same as Machine A

Initially, \$1 = 0, \$2 = 0, \$3 = 0, and \$4 = 400.

- (a) Re-write the code segment above *with minimal changes* so that it gets correctly executed in Machine A *with minimal latency*. You can either insert nop instructions or reorder instructions as needed.
- (b) Fill the table below with the timeline of the first loop iteration of the code segment in Machine A.

Instruction	Clock cycle number																		
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

- (c) Calculate the number of cycles it takes to execute the code segment on Machine A. Show your work in the box.
- (d) Fill the table below with the timeline of the first loop iteration of the code segment in Machine B.

Instruction	Clock cycle number																		
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
lw \$1, 0(\$4)																			
lw \$2, 400(\$4)																			
add \$3, \$1, \$2																			
sw \$3, 0(\$4)																			
sub \$4, \$4, #4																			
bnez \$4, Loop																			

- (a) Calculate the number of cycles it takes to execute the code segment on Machine B. Show your work in the box.

Tomasulo's Algorithm

(Q7)

Consider an in-order fetch, out-of-order dispatch, and in-order retirement execution engine that employs Tomasulo's algorithm. This engine has the following characteristics:

- The engine has four main pipeline stages: Fetch (F), Decode (D), Execute (E), and Write-back (W).
- The engine can fetch one instruction per cycle, decode one instruction per cycle, and write back the result of one instruction per cycle.
- The engine has two execution units: 1) an adder to execute ADD instructions and 2) a multiplier to execute MUL instructions.
- The execution units are fully pipelined. The adder has two stages (E1-E2), and the multiplier has four stages (E1-E2-E3-E4). Execution of each stage takes one cycle.
- The adder has a two-entry reservation station, and the multiplier has a three-entry reservation station.
- An instruction always allocates the first available entry of the reservation station (in top-to-bottom order) of the corresponding execution unit.
- Full data forwarding is available, i.e., during the last cycle of the E stage, the tags and data are broadcast to the reservation station and the Register Alias Table (RAT).
- For example, an ADD instruction updates the reservation station entries of the dependent instructions in the E2 stage. So, the updated value can be read from the reservation station entry in the next cycle. Therefore, a dependent instruction can potentially begin its execution in the next cycle (after E2).
- The multiplier and adder have separate output data buses, which allow both the adder and the multiplier to update the reservation station and the RAT in the same cycle.
- An instruction continues to occupy a reservation station slot until it finishes the Write-back (W) stage. The reservation station entry is deallocated after the Write-back (W) stage.

7.1 Problem Definition

The processor is about to fetch and execute five instructions. Assume the reservation stations (RS) are all initially empty, and the initial state of the register alias table (RAT) is given below in Figure (a). Instructions are fetched, decoded, and executed as discussed in class. At some point during the execution of the five instructions, a snapshot of the state of the RS and the RAT is taken. Figures (b) and (c) show the state of the RS and the RAT at the snapshot time. A dash (–) indicates that a value has been cleared. A question mark (?) indicates that a value is unknown to you.

7.2 Questions

7.2.1 Dataflow Graph

Based on the information provided above, identify the instructions and provide the dataflow graph below for the instructions that have been fetched. Please appropriately connect the nodes using edges and specify the direction of each edge. Label each edge with the destination architectural register and the corresponding Tag.

7.2.2 Program Instructions

Fill in the blanks below with the five-instruction sequence in program order. There can be more than one correct ordering. Please provide only one correct ordering. When referring to registers, please use their architectural names (R0 through R9). Place the register with the smaller architectural name on the left source register box. For example, $\text{ADD } R8 \leftarrow R1, R5$.

		←		
		←		
		←		
		←		
		←		

Reg	Valid	Tag	Value
R0	1	–	1900
R1	1	–	82
R2	1	–	1
R3	1	–	3
R4	1	–	10
R5	1	–	5
R6	1	–	23
R7	1	–	35
R8	1	–	61
R9	1	–	4

(a) Initial state of the RAT

Reg	Valid	Tag	Value
R0	1	?	1900
R1	1	?	82
R2	1	?	1
R3	1	?	45
R4	0	A	?
R5	0	F	?
R6	1	?	23
R7	1	?	35
R8	0	L	?
R9	0	B	?

(b) State of the RAT at the snapshot time

ID	V	Tag	Value	V	Tag	Value
–	–	–	–	–	–	–
L	1	?	82	1	?	1

+

ID	V	Tag	Value	V	Tag	Value
F	1	?	45	1	?	1
A	0	F	?	1	?	10
B	1	?	23	1	?	45

×

(c) State of the RS at the snapshot time

We define the *SIMD utilization* of a program that runs on a GPU as the fraction of SIMD lanes that are kept busy with *active threads* during the run of a program.

The following code segments are run on a GPU. We assume that (1) A resides in memory and is shared by all threads, (2) s resides in a register and is private to each thread, and (3) the code segments are correct (i.e., do not think about any correctness issues when answering this question).

A warp in the GPU consists of 32 threads, and there are 32 SIMD lanes in the GPU. Each thread executes a **single iteration** of the outermost loop (with index i). Assume that the data values of the array A are already in vector registers so there are no memory loads and stores in this program. (Hint: Notice that there are 4 instructions in each iteration of the outermost loop of both code segments.)

```
s = 1;
for (i = 0; i < 1024; i++) {
    for (j = 0; j < 10; j++) { // Inst. 1
        if (i % (2 * s) == 0) // Inst. 2
            A[i] += A[i + 1]; // Inst. 3
        s = s << 1;           // Inst. 4
    }
}
```

Code Segment 1

```
s = 512;
for (i = 0; i < 1024; i++) {
    for (j = 0; j < 10; j++) { // Inst. 1
        if (i < s)              // Inst. 2
            A[i] += A[i + s]; // Inst. 3
        s = s >> 1;           // Inst. 4
    }
}
```

Code Segment 2

Please answer the following questions.

- How many warps does it take to execute these code segments?
- What is the SIMD utilization of the first iteration of the inner loop ($j = 0$) for Code Segment 1? Show your work. (Hint: The warp scheduler does *not* issue instructions when no thread is active).
- What is the SIMD utilization of the first iteration of the inner loop ($j = 0$) for Code Segment 2? Show your work. (Hint: The warp scheduler does *not* issue instructions when no thread is active).
- What is the SIMD utilization of any iteration of the inner loop ($0 \leq j < 10$) for Code Segment 1? Show your work. (Hint: Derive an analytical expression, which may be piecewise).
- What is the SIMD utilization of any iteration of the inner loop ($0 \leq j < 10$) for Code Segment 2? Show your work. (Hint: Derive an analytical expression, which may be piecewise).
- Is there any iteration ($0 \leq j < 10$) where both code segments have the same utilization? Explain your reasoning.
- Which code is expected to run faster on a GPU? Explain your reasoning.

Branch Prediction

(Q9)

You are given the following piece of code that iterates through two large arrays, *j* and *k*, each populated with completely (i.e., truly) random positive integers. The code has five branches (labeled B1, B2, B3, B4, and B5). When we say that a branch is *taken*, we mean that the code inside the curly brackets is executed. Assume that the code is run to completion without any errors or interruptions (i.e., there are no exceptions). For the following questions, assume that this is the only block of code that will ever be run on the machines, and that the loop condition branch is resolved first in the iteration (i.e., the if statements execute only *after* resolving the loop condition branch).

```
1  for(int i = 0; i < 1000; i++) { //B1
2                                     //TAKEN PATH for B1
3      if (i % 2 == 0) { //B2
4          j[i] = k[i] * i; //TAKEN PATH for B2
5      }
6      if (i < 250) { //B3
7          j[i] = k[i] - i; //TAKEN PATH for B3
8      }
9      if (i < 500) { //B4
10         j[i] = k[i] + i; //TAKEN PATH for B4
11     }
12     if (i >= 500) { //B5
13         j[i] = k[i] / i; //TAKEN PATH for B5
14     }
15 }
```

Listing 1: Application to evaluate.

You are given three machines whose components are identical in every way, except for their branch predictors.

- Machine A uses an always-taken branch predictor.
- Machine B uses one single-level global two-bit saturating counter branch predictor *shared by all branches*, which starts at Weakly Taken (2'b10).
- Machine C uses a *per-branch* two-bit saturating counter as its branch predictor. All counters start at Weakly Not Taken (2'b01).

The saturating counter values are as follows:

- 2'b00 - Strongly Not Taken
- 2'b01 - Weakly Not Taken
- 2'b10 - Weakly Taken
- 2'b11 - Strongly Taken

Answer the following questions:

1. What is the branch misprediction rate when the above piece of code runs on Machine A? Show your work.
2. What is the branch misprediction rate when the above piece of code runs on Machine B? Show your work.
3. What is the branch misprediction rate when the above piece of code runs on Machine C? Show your work.

Caches

(Q10)

You are trying to reverse-engineer the characteristics of a cache in a system, so that you can design a more efficient, machine-specific implementation of an algorithm you are working on. To do so, you have come up with three sequences of memory accesses to various *bytes* in the system in an attempt to determine the following four cache characteristics:

- Cache block size (8, 16, 32, 64, or 128 B).
- Cache associativity (2-, 4-, or 8-way).
- Cache replacement policy (LRU or FIFO).
- Cache size (4 or 8 KiB).

The only statistic that you can collect on this system is *cache hit rate* after performing each sequence of memory accesses. Here is what you observe:

Sequence	Addresses Accessed (Oldest → Youngest)								Hit Rate
1.	0	16	24	25	1024	255	1100	305	2/8
2.	31	65536	65537	131072	262144	8	305	1060	3/8
3.	262145	65536	4						2/3

Assume that the cache is initially empty at the beginning of the first sequence, but not at the beginning of the second and third sequence. The sequences are executed back-to-back, i.e., no other accesses take place in between sequences. Thus, **at the beginning of the second sequence, the contents are the same as at the end of the first sequence. At the beginning of the third sequence, the contents are the same as at the end of the second sequence.**

Based on what you observe, what are the following characteristics of the cache? Explain to get points.

- (a) Cache block size (8, 16, 32, 64, or 128 B)?
- (b) Cache associativity (2-, 4-, or 8-way)?
- (c) Cache replacement policy (LRU or FIFO)?
- (d) To identify the cache size (4 or 8 KiB), you can access two addresses right after sequence 3 (i.e., the contents are the same as at the end of the third sequence) and measure the cache hit rate. Which two addresses would you choose? Explain your answer (there may be several correct answers).

BONUS: Prefetching

(Q11)

A runahead execution processor is designed with an unintended hardware bug: every other instruction in runahead mode is dropped by the processor after the fetch stage. Recall that the runahead mode is the speculative processing mode where the processor executes instructions solely to generate prefetch requests. All other behavior of the runahead mode is exactly as we described in lectures. When a program is executed, which of the following scenarios could happen compared to a runahead processor without the hardware bug and why? Circle YES if there is a possibility to observe the described behavior and explain in the box (either if you answer YES or NO). Assume that the program has no bug in it and executes correctly on the processor without the hardware bug.

(a) The buggy runahead processor finishes the program *correctly* and *faster* than the non-buggy runahead processor.

YES NO

Why?

(b) The buggy runahead processor finishes the program *correctly* and *slower* than the nonbuggy runahead processor.

YES NO

Why?

(c) The buggy runahead processor executes the program *incorrectly*.

YES NO

Why?

BONUS: Systolic Arrays

(Q12)

A systolic array consists of 4x4 Processing Elements (PEs), interconnected as shown in Figure 1. The inputs of the systolic array are labeled as H_0, H_1, H_2, H_3 and V_0, V_1, V_2, V_3 . Figure 2 shows the PE logic, which performs a multiply and accumulate MAC operation and saves the result to an internal register (*reg*). Figure 2 also shows how each PE propagates its inputs. We make the following assumptions:

- The latency of each MAC operation is one cycle.
- The propagation of the values from i_0 to o_0 , and from i_1 to o_1 , takes one cycle.
- The initial values of all internal registers is zero.

Your goal is to use the example systolic array shown in Figure 1 to perform the convolution ($\otimes$) of a 3x3 image (matrix $I_{3 \times 3}$) with four 2x2 filters (matrices $A_{2 \times 2}, B_{2 \times 2}, C_{2 \times 2}$, and $D_{2 \times 2}$), to obtain four 2x2 outputs (matrices $W_{2 \times 2}, X_{2 \times 2}, Y_{2 \times 2}$, and $Z_{2 \times 2}$):

$$\begin{matrix} I_{00} & I_{01} & I_{02} \\ I_{10} & I_{11} & I_{12} \\ I_{20} & I_{21} & I_{22} \end{matrix} \otimes \begin{matrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{matrix} = \begin{matrix} W_{00} & W_{01} \\ W_{10} & W_{11} \end{matrix}$$

$$\begin{matrix} I_{00} & I_{01} & I_{02} \\ I_{10} & I_{11} & I_{12} \\ I_{20} & I_{21} & I_{22} \end{matrix} \otimes \begin{matrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{matrix} = \begin{matrix} X_{00} & X_{01} \\ X_{10} & X_{11} \end{matrix}$$

$$\begin{matrix} I_{00} & I_{01} & I_{02} \\ I_{10} & I_{11} & I_{12} \\ I_{20} & I_{21} & I_{22} \end{matrix} \otimes \begin{matrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{matrix} = \begin{matrix} Y_{00} & Y_{01} \\ Y_{10} & Y_{11} \end{matrix}$$

$$\begin{matrix} I_{00} & I_{01} & I_{02} \\ I_{10} & I_{11} & I_{12} \\ I_{20} & I_{21} & I_{22} \end{matrix} \otimes \begin{matrix} D_{00} & D_{01} \\ D_{10} & D_{11} \end{matrix} = \begin{matrix} Z_{00} & Z_{01} \\ Z_{10} & Z_{11} \end{matrix}$$

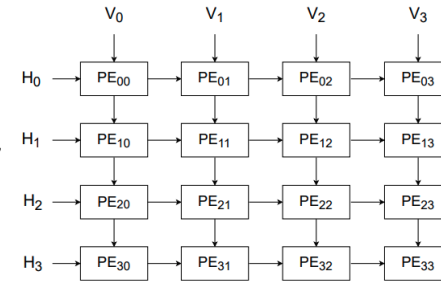


Figure 1: PE array

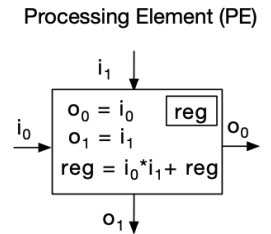


Figure 2: Processing Element (PE)

As an example, the convolution of the matrix $I_{3 \times 3}$ with the filter $A_{2 \times 2}$ is computed as follows:

- $W_{00} = I_{00} * A_{00} + I_{01} * A_{01} + I_{10} * A_{10} + I_{11} * A_{11}$
- $W_{01} = I_{01} * A_{00} + I_{02} * A_{01} + I_{11} * A_{10} + I_{12} * A_{11}$
- $W_{10} = I_{10} * A_{00} + I_{11} * A_{01} + I_{20} * A_{10} + I_{21} * A_{11}$
- $W_{11} = I_{11} * A_{00} + I_{12} * A_{01} + I_{21} * A_{10} + I_{22} * A_{11}$

You should compute the four convolutions in the minimum possible number of cycles. Fill the following table with:

1. The input elements (from matrices $I_{3 \times 3}, A_{2 \times 2}, B_{2 \times 2}, C_{2 \times 2}$, and $D_{2 \times 2}$) in the correct input ports of the systolic array (H_0, H_1, H_2, H_3 and V_0, V_1, V_2, V_3). (Hint: If necessary, an input element can be concurrently streamed into several input ports of the array.)
2. The output values and the corresponding PE where the output elements (of matrices $W_{2 \times 2}, X_{2 \times 2}, Y_{2 \times 2}$, and $Z_{2 \times 2}$) are generated.

cycle	H0	H1	H2	H3	V0	V1	V2	V3	PE ₀₀	PE ₀₁	PE ₀₂	PE ₀₃	PE ₁₀	PE ₁₁	PE ₁₂	PE ₁₃	PE ₂₀	PE ₂₁	PE ₂₂	PE ₂₃	PE ₃₀	PE ₃₁	PE ₃₂	PE ₃₃
0																								
1																								
2																								
3																								
4																								
5																								
6																								
7																								
8																								
9																								
10																								
11																								
12																								
13																								
14																								
15																								

Fill the blanks only with relevant information.

Digital Design & Computer Arch.

Problem Solving V

Prof. Onur Mutlu

ETH Zürich
Spring 2026
24 July 2026